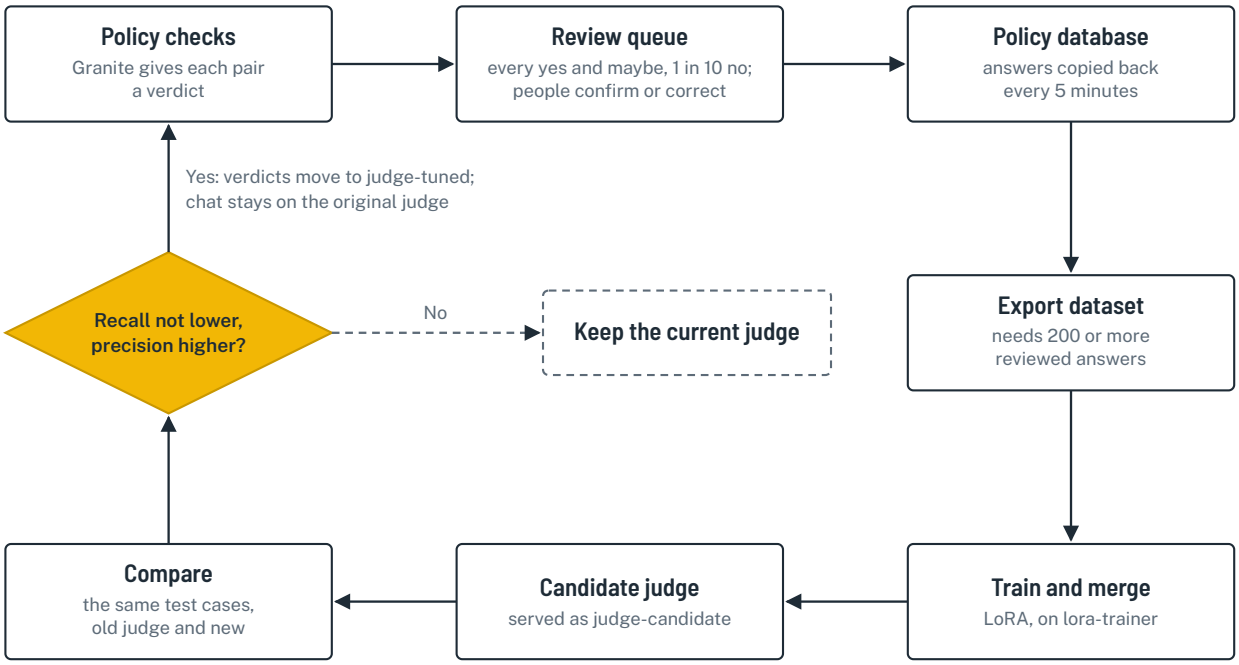
Review queue and judge training

Let people check answers, then teach the judge from them.

For: Administrators, reviewers and engineers

One chapter of the handbook. The full handbook, with every chapter, is handbook.pdf in the same folder.

People check the judge's answers in a review queue (Argilla). Their corrections are saved and, once there are enough, used to fine-tune the judge. Nothing leaves your servers: everything runs on the sealed `backend` network, with telemetry and internet access turned off.



The review and training loop. A new judge replaces the old one only if it scores better.

For the administrator

What you get

- **Review queue**: one card per "this section may be affected" answer. The reviewer sees the old and new wording, the other policy's section and the program's answer, and presses Submit.
- **Training**: after a few hundred reviewed cards, an engineer can teach the judge from them. A new judge only replaces the old one if it scores better on your test cases.

Start the review queue

1. In `.env`, fill in the "Review queue (Argilla)" block. Make each secret with `openssl rand -hex 16` (use letters and digits only).
2. Start it: `docker compose --profile review up -d`. The first start takes a few minutes.
3. Ask whoever runs the web proxy to add the Argilla address (see "Engineer: proxy" below).
4. Open `http://<your-server>:6900` and sign in as the admin user from `.env`.

Add a reviewer

Run once per person (change the name, username and password):

```
docker compose exec argilla python -m argilla_server database users create \
  --first-name "Ana" --username ana --password "a-long-password" --role annotator --workspace
policy-review
```

Give them the address, their username and password. A reviewer only sees their own cards. To add a reviewer who can also see everyone's work, use `--role admin` instead of `annotator`.

What reviewers do

Reviewers need no training. The first screen shows the instructions in plain words. For each card:

1. Read the old and new wording, then the section that may be affected.
2. The program's answer is already filled in. If it is right, press **Submit**.
3. If it is wrong, change the answer, pick the kind of impact, add a short note saying why, and press **Submit**.
4. **Can't tell** is allowed. **Discard** skips a card. Nothing is ever lost or published.

Ask reviewers to correct the program when it is wrong, not to agree by default. Corrections are the most useful part.

How answers come back

The sync job in policy-api sends new answers to the queue and copies submitted answers back into the policy database (`reviewer_decision`, `reviewer_impact_type`, `reviewer_note`, `reviewed_by`, `reviewed_at`). A card that is only a draft or was discarded is ignored.

Turn the sync on by setting `ARGILLA_URL=http://argilla:6900` in `.env` (with `ARGILLA_API_KEY` already set) and restarting policy-api. It runs every 5 minutes (`REVIEW_SYNC_SECONDS`) as a job in the job queue. It sends every "yes" and "maybe" from finished checks, plus 1 in 10 of the "no" answers (`REVIEW_SAMPLE_NO`), because reviewers also need to catch what the program missed. Each card shows the clause exactly as the program saw it, even if the policy has changed since.

When is there enough?

Training refuses to start below **200 reviewed answers** and tells you how many more it needs. More is better: aim for 500 or more, covering at least 5 different changed policies, and include plenty of corrections, not only agreements. Check the count any time:

```
docker compose --profile train run --rm lora-trainer export_dataset.py --out /out/data
```

It prints the counts and writes nothing until there is enough.

Back up and delete

The queue's data lives in the `argilla-pgdata`, `argilla-esdata` and `argilla-data` volumes. Back up `argilla-pgdata` with the policy database. The index (`argilla-esdata`) can be rebuilt. Reviewer answers are also copied into the policy database, which is the record that counts.

Engineer

Services (all on `backend`, none publish a port)

Service	What	Profile
`argilla`, `argilla-worker`	Argilla v2.8.0 server and background worker (UI port 6900 inside the network)	review
`argilla-db`	Postgres 16.6, Argilla's own data, kept apart from the policy database	review
`argilla-es`	Elasticsearch 8.17.0, search index	review
`argilla-redis`	Redis 7.4.2, job queue	review
`lora-trainer`	exports data, trains and merges (image built from `training/Dockerfile`)	train
`judge-candidate`	vLLM serving the merged model as `judge-candidate`	train

Versions are set in `.env` (`ARGILLA_TAG`, `ARGILLA_ES_TAG`, `ARGILLA_REDIS_TAG`, `ARGILLA_PG_TAG`). The Python client `argilla==2.8.0` in `policy_api/requirements.txt` must match `ARGILLA_TAG`. Telemetry is off: `HF_HUB_DISABLE_TELEMETRY=1`, `HF_HUB_OFFLINE=1`, `ARGILLA_ENABLE_TELEMETRY=0`.

`argilla` refuses to start if `ARGILLA_ADMIN_PASSWORD` or `ARGILLA_API_KEY` is empty, and the password must be 8+ characters. The first start creates the owner account and workspace from `.env`. Restarting is safe (the account is not created twice). Its healthcheck uses `/api/docs`: in v2.8.0 `/api/v1/status` returns 422 even when healthy.

Proxy: expose the Argilla UI

Reviewers reach Argilla through the existing proxy. Add this `server` block to `proxy/nginx.conf` (next to the Open WebUI one), and publish the port on the `proxy` service: `- "${BIND_ADDR:-127.0.0.1}:6900:6900"`.

```
# Argilla (review queue)
server {
  listen 6900;
  client_max_body_size 25m;
  location / {
    set $argilla http://argilla:6900;
    proxy_pass $argilla;
    proxy_http_version 1.1;
    proxy_set_header Host $host;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
  }
}
```

policy-api talks to Argilla directly on the backend network, never through the proxy: `ArgillaReview(api_url="http://argilla:6900", api_key=ARGILLA_API_KEY, workspace=ARGILLA_WORKSPACE)` (`policy_api/app/review.py`). Calls are synchronous; call them with `asyncio.to_thread`.

What goes in and out

- `push(items)`: one record per judgement, record id = `str(judgements.id)`. Cards already in the queue are skipped, so a reviewer's work is never overwritten. The program's answer is pre-filled.
- `pull_submitted(since=None)`: only responses with status `submitted`. If two people answered one card, the latest wins. `no` always comes back with impact type `none`. Pass the newest `reviewed_at` you have stored as `since` to pull only new answers.
- Training reads the same rows from Postgres: `reviewer_decision` plus `prompt`, so `prompt` must be the exact text the judge saw (`render_pair`).

Train

All commands run from the repo root. Granite runs on its own server, so the GPUs here only need to hold the training job: give `lora-trainer` (and later `judge-candidate`) GPUs in `compose.gpu.yaml`, like the others:

```
  lora-trainer:
    deploy: {resources: {reservations: {devices: [{driver: nvidia, device_ids: ["0"],
capabilities: [gpu]}]}}}
  judge-candidate:
    deploy: {resources: {reservations: {devices: [{driver: nvidia, device_ids: ["1","2"],
capabilities: [gpu]}]}}}
```

```
docker compose --profile train build lora-trainer          # once; needs internet, or build
elsewhere and `docker load`
docker compose --profile train run --rm lora-trainer export_dataset.py --out /out/data
docker compose --profile train run --rm lora-trainer train_lora.py \
    --train /out/data/train.jsonl --eval /out/data/holdout.jsonl --base /models/gpt-oss-20b --out
/out/adapter
docker compose --profile train run --rm lora-trainer merge_adapter.py \
    --base /models/gpt-oss-20b --adapter /out/adapter --out /out/merged
```

Outputs are in `training/out/` (not in git: they contain policy text). `training/README.md` has the settings and the details.

Evaluate and promote

The rule: **switch only if recall is not lower and precision is higher** on the same test cases.

1. Start the candidate: `docker compose --profile train up -d judge-candidate` (waits for the model to load).
2. Run the same cases against both judges:

```
docker compose exec policy-api python -m app.evaluate /eval/cases.json > training/out/base.json
docker compose exec -e JUDGE_URL=http://judge-candidate:8000/v1 -e JUDGE_MODEL=judge-candidate \
    policy-api python -m app.evaluate /eval/cases.json > training/out/candidate.json
python3 training/compare_eval.py training/out/base.json training/out/candidate.json
```

`app.evaluate` already reads `JUDGE_URL` and `JUDGE_MODEL` from the environment, so it needs no change. Keep `eval/cases.json` free of the policies used for training (it is a separate answer key). 3. If `compare_eval.py` prints `RESULT: the new model is better` (exit code 0), switch.

Switch. The tuned model gets its own service, `judge-tuned`. Only the policy checks' verdicts move to it; the chat in Open WebUI stays on Granite, so a model trained only to give verdicts never has to hold a conversation.

```
docker compose --profile train stop judge-candidate
mv training/out/merged models/judge-tuned-2026-11        # any name; keep one folder per version
```

In `.env` (and assign it GPUs with `python3 scripts/setup_gpus.py`, answering for `judge-tuned`):

```
TUNED_JUDGE_DIR=judge-tuned-2026-11
JUDGE_URL=http://judge-tuned:8000/v1
JUDGE_MODEL=judge-tuned
```

```
docker compose --profile tuned up -d judge-tuned      # wait until `docker compose ps judge-
tuned` is healthy
docker compose up -d policy-api
```

Run one real check from Open WebUI. Every verdict records the model that gave it, so the `judgements` table shows which checks used the tuned judge.

Roll back (seconds, no model reload): delete or comment out the `JUDGE_URL` and `JUDGE_MODEL` lines in `.env`, then `docker compose up -d policy-api`. Stop `judge-tuned` when you no longer need it.

The Granite server is never replaced; the tuned model only takes over the verdicts.

Known limits

- The judge is now Granite 4.2 30B, but these scripts still fine-tune gpt-oss-20b. A tuned gpt-oss-20b can still be tried as `judge-candidate` and must beat Granite on the test cases to replace it. Tuning Granite itself needs `train_lora.py` changed for a dense bf16 model (no `Mxfp4Config`, LoRA on attention and MLP layers) and is not done yet.
- vLLM cannot apply LoRA to gpt-oss mixture-of-experts layers, so training is attention-only by default and the adapter is always merged into a normal model folder before serving.
- The merged model is bf16: about 42 GB for 20b and about 240 GB for 120b, so `judge-candidate` needs correspondingly more GPU memory than the shipped 4-bit model. Re-quantizing is optional and not done here.
- Training on reviewer answers teaches the judge their habits, including mistakes. The test cases are the check; never skip the promotion rule.